

## Introduction:

"Hi, I'm Akhil Gupta. I'm an engineering leader with over 17 years of experience building enterprise software platforms, and the last 9+ years leading global engineering teams across banking, AI platforms, data governance, and enterprise SaaS.

Currently, I'm a Vice President of Technology at JPMorgan Chase, where I lead architecture and delivery of an enterprise Agentic AI platform — we've built a chain-of-agents system with intent, orchestrator, and domain-specific agents that turns marketing campaign onboarding into a prompt-driven workflow, significantly reducing manual effort. I've also worked on real-time fraud detection using AI and on-device inference to improve latency and compliance.

Before this, I spent seven years at IBM leading engineering and product for Watson Knowledge Catalog, where I architected a data lineage and governance platform that improved traceability by 70%, and delivered LLM-powered compliance and explainability features. I've also worked at OpenText leading a cloud-native GenAI SaaS platform.

Across my career, I've focused on distributed systems, GenAI and agentic AI architectures, and building high-performing teams — while staying close to product and business goals. I'm passionate about turning emerging AI capabilities into practical, governed, enterprise-grade solutions, and I'm excited to bring that experience to this role."

## Work at JPMC:

At JPMorgan, marketing campaign onboarding was a heavily manual process — business teams had to work through multiple systems, forms, and approval chains just to set up a single campaign. This slowed down launches and consumed a lot of engineering and operations bandwidth for what was essentially repetitive, rules-based work."

### What I did (Architecture & Approach):

"I led the architecture and delivery of an Agentic AI platform that converted this entire onboarding process into a prompt-driven workflow. Instead of navigating multiple forms and systems, a business user could describe what they needed in natural language, and the system would handle the rest.

To make this reliable and enterprise-grade — not just a chatbot wrapper — I designed a **chain-of-agents architecture** with three layers:

1. **Intent Agent** – interprets the user's natural language request and figures out what they're actually trying to do.
2. **Orchestrator Agent** – takes that intent and plans out the workflow: which steps need to happen, in what order, and which downstream systems need to be involved.
3. **Domain-specific Service Agents** – specialized agents that actually execute tasks against specific enterprise systems — for example, one might handle campaign metadata, another might handle approvals, another might integrate with the marketing backend.

This separation of concerns was intentional — it meant we could add new capabilities or integrate new systems by adding a new domain agent, without having to touch the core orchestration logic. That made the platform modular and extensible."

**Why this mattered (Governance & Security):**

"Because this was a regulated banking environment, it couldn't just be 'AI does whatever it wants.' Every agent operated within defined boundaries, actions were auditable, and the orchestrator enforced guardrails around what each domain agent could and couldn't do. So we got the speed and flexibility of AI-driven automation, but with the security and governance controls that JPMorgan requires."

**Outcome:**

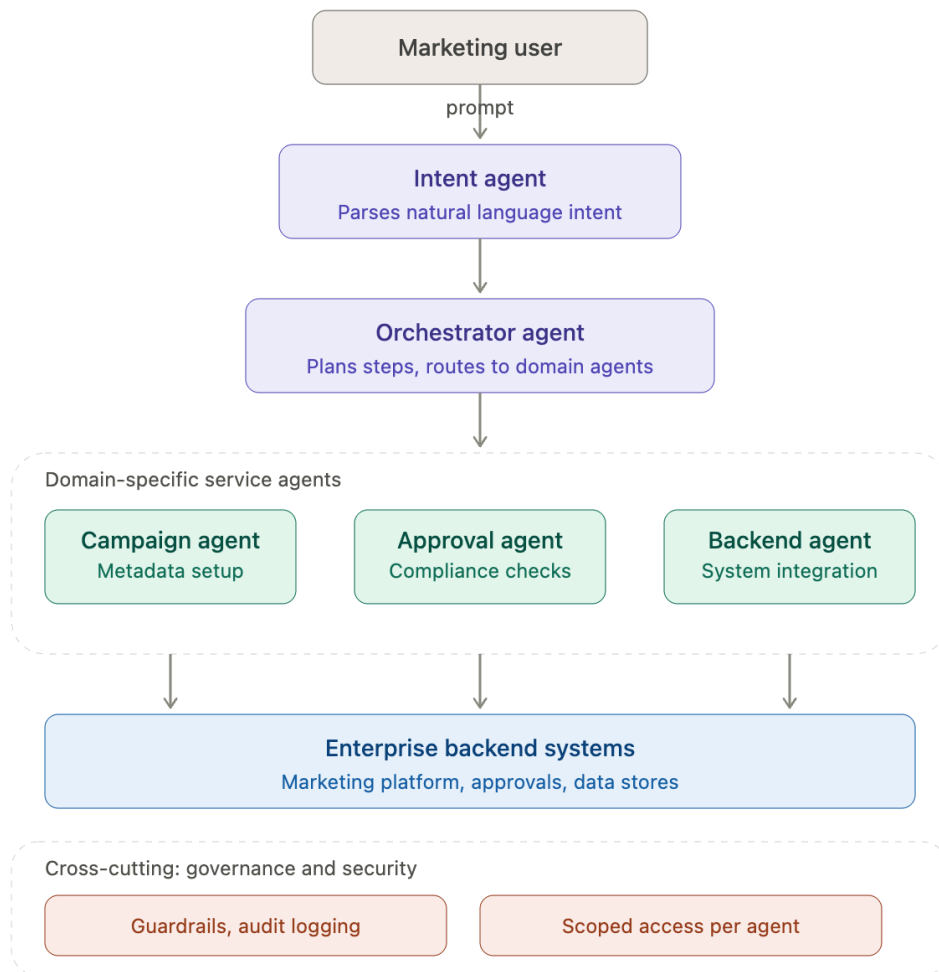
"The result was a significant reduction in manual effort for campaign onboarding and faster campaign launches — turning a multi-step, multi-system manual process into a single conversational workflow."

**Why chain-of-agents vs. a single monolithic LLM call?** (Modularity, easier debugging, clearer ownership per domain, easier to govern/audit each agent's scope)

**How you handled hallucination/reliability risk** (guardrails, validation steps, human-in-the-loop for sensitive actions if applicable)

**How you measured success** (time saved, campaigns launched faster, reduction in manual tickets/effort)

**Team/stakeholder management** (how you coordinated with product, compliance, and other engineering teams)



### 1. Entry point — user request

A marketing team member types a natural language request (e.g., "Set up a campaign for our Q3 credit card promo targeting existing customers").

### 2. Intent agent

This is the first stop. It parses the free-form prompt and converts it into a structured intent — figuring out *what* the user actually wants (create campaign, modify approval, check status, etc.) and extracting key parameters. This decouples "understanding the user" from "executing the workflow," so the rest of the system doesn't have to deal with raw natural language.

### 3. Orchestrator agent

This is the brain of the workflow. Once intent is structured, the orchestrator plans the sequence of steps needed to fulfill it — deciding which domain agents to call, in what order, and how to handle dependencies between them (e.g., campaign metadata must be created before an approval request can be triggered). It's essentially a dynamic workflow planner rather than a hardcoded pipeline, which is what makes the platform extensible.

#### 4. Domain-specific service agents

Each agent owns a narrow, well-defined responsibility and talks to specific enterprise systems:

- **Campaign agent** — handles campaign metadata and setup
- **Approval agent** — runs compliance/policy checks
- **Backend agent** — integrates with downstream marketing and data systems

This is the key architectural decision — by isolating responsibilities per agent, you can add a new capability (say, a budget-check agent) without touching the orchestrator's core logic. It also makes testing, debugging, and auditing far easier, since each agent's scope is bounded.

#### 5. Enterprise backend systems

The domain agents ultimately read/write against real systems — the marketing platform, approval workflows, data stores — so the AI layer sits on top of, not instead of, existing enterprise infrastructure.

#### 6. Cross-cutting governance and security layer

This runs alongside every layer rather than being a separate step: every agent action is logged for audit, and each agent has scoped access — it can only call the specific systems/actions it's authorized for. This is what makes the platform viable in a regulated banking environment — it's not open-ended agentic autonomy, it's governed automation.

### Real-Time Fraud Detection for Card Transactions

Context / Problem:

"Card transaction fraud needs to be caught in real time — if you flag a fraudulent transaction after the fact, the damage is already done. The challenge was designing a system that could evaluate risk on every transaction within milliseconds, at the scale JPMorgan operates, without introducing friction for legitimate customers."

What I did (Architecture & Approach):

"I defined the architecture for real-time fraud detection services that combined AI-based risk scoring with distributed systems design. The core challenge wasn't just building a good model — it was building the surrounding system so that inference could happen fast enough to sit in the critical path of a live transaction, and reliably enough to handle the transaction volume of a major bank.

That meant thinking through things like:

- How to serve model predictions with very low latency
- How to keep the system resilient and horizontally scalable under high transaction throughput

- How to feed the model fresh signals (transaction patterns, device/location context, etc.) without slowing down the decision"

Why this mattered:

"Fraud detection is a direct trade-off between security and customer experience. Too aggressive, and you block legitimate customers; too lenient, and fraud slips through. So the architecture had to support fast, accurate decisions while integrating cleanly with existing risk and transaction infrastructure."

Outcome:

"This gave the bank a scalable, low-latency fraud detection capability that could evaluate risk on card transactions in real time, strengthening security without adding friction to genuine transactions."

### **Project: On-Device AI Inference using ExecuTorch**

Context / Problem:

"A lot of AI inference in banking apps traditionally happens server-side — the app sends data to a backend, a model runs, and a result comes back. But that has three downsides: latency, privacy exposure (sensitive data leaving the device), and regulatory complexity around where and how customer data is processed."

What I did (Architecture & Approach):

"I introduced on-device AI inference using ExecuTorch — a framework that lets you run AI models directly on the user's device (like a phone) instead of a server. This meant taking models that would normally run in the cloud and optimizing them to run efficiently on-device, within the compute and memory constraints of a mobile environment."

This is a meaningfully different engineering problem than server-side AI — you're constrained by battery, memory, and processor limits, so the model and the inference pipeline both need to be lightweight and efficient."

Why this mattered:

"Three concrete benefits came out of this:

1. Latency — no network round-trip means faster results for the user.
2. Privacy — sensitive data never has to leave the device, which is a big deal in banking.
3. Regulatory compliance — processing data on-device sidesteps a lot of the complexity around cross-border data transfer and data residency rules that come with server-side processing."

Outcome:

"This gave the platform a faster, more private, and more compliant way to deliver AI capabilities directly to users, and it's a pattern that can extend to other latency- or privacy-sensitive use cases going forward."

# Interview Preparation — 100 Questions & Answers

Based on Akhil Gupta's Resume (VP Technology, AI Platforms — JPMorgan Chase)

---

## Section 1: Background, Career Journey & Summary (Q1–10)

**Q1. Walk me through your career journey.** A: I've spent 17+ years building enterprise software, starting as a Systems Engineer at TCS working on Java/Spring web applications, then moving through eGain (knowledge management), Pramati (microservices/resiliency), and ADP (HR platform architecture). Over the last 9+ years, I've moved into engineering leadership — 7 years at IBM leading Watson Knowledge Catalog, then OpenText leading a GenAI SaaS platform, and now VP of Technology at JPMorgan Chase leading AI Platforms.

**Q2. Why did you move from a hands-on engineer to engineering leadership?** A: As I built deeper technical expertise in distributed systems and architecture, I found I could create more impact by shaping technical strategy, mentoring teams, and aligning engineering execution with business goals — rather than only writing code myself. Leadership let me multiply my impact across teams.

**Q3. What's the common thread across your career?** A: Enterprise-scale platforms — I've consistently worked on systems that need to be reliable, secure, and governed, whether that's HR data at ADP, knowledge catalogs at IBM, or now financial services AI at JPMorgan. The other thread is staying close to emerging technology — I moved early into GenAI and now agentic AI.

**Q4. Why did you choose to specialize in AI/data platforms?** A: Data and AI sit at the center of enterprise decision-making. I saw early at IBM, working on Watson Knowledge Catalog, how governance and trust in data directly determines whether AI can be adopted safely — that's shaped my focus since.

**Q5. What motivated your move to JPMorgan?** A: The opportunity to apply agentic AI at enterprise scale in a highly regulated, high-stakes environment — banking demands both innovation and rigor, and that combination is where I do my best work.

**Q6. How do you describe yourself as an engineering leader in one sentence?** A: Someone who bridges deep technical architecture with pragmatic delivery — I can go deep into distributed systems design while also managing stakeholders, roadmaps, and people.

**Q7. What are you most proud of in your career so far?** A: Leading the Agentic AI platform at JPMorgan — taking a manual, multi-system process and turning it into a governed, prompt-driven workflow. It combined architecture, AI innovation, and enterprise-grade security in a way that had real business impact.

**Q8. What's a career decision you'd make differently?** A: (Personalize, but a safe framing:) Earlier in my career I could have moved into architecture and leadership roles sooner — I spent longer than necessary in pure execution roles before stepping into ownership positions.

**Q9. Why are you looking for a new opportunity now?** A: (Tailor to real reason — growth, scope, technology focus, etc. Keep positive and forward-looking, not critical of current employer.)

**Q10. What are you looking for in your next role?** A: A role where I can continue leading AI-driven platform architecture at scale, with the opportunity to shape technical strategy and build/mentor high-performing teams — ideally in a domain where AI has significant real-world impact.

---

## **Section 2: Leadership & Engineering Management (Q11–25)**

**Q11. How do you approach building and leading engineering teams?** A: I focus on three things: clear technical vision so the team knows the "why," strong ownership boundaries so people know what they're accountable for, and consistent mentoring so the team grows technically over time. I also prioritize psychological safety — people need to be comfortable raising concerns.

**Q12. How large are the teams you've managed?** A: I've managed teams up to 8 engineers directly (OpenText) and led larger cross-functional and global engineering teams across my IBM and JPMorgan tenures, working across multiple pods/squads.

**Q13. How do you balance hands-on architecture work with people leadership?** A: I stay engaged in architecture reviews and key design decisions — that's where I add the most unique value — while delegating day-to-day execution to leads and giving them ownership. I protect time for both.

**Q14. How do you mentor engineers?** A: Through regular architecture reviews where I ask probing questions rather than giving answers, pairing junior engineers with more complex design problems, and creating space for them to present their own designs to stakeholders.

**Q15. Tell me about a time you had to manage a difficult stakeholder.** A: (Use a real example — e.g., balancing compliance/security requirements from risk teams against product speed asks from business stakeholders in the Agentic AI platform work. Focus on how you found a governed middle path.)

**Q16. How do you handle disagreements within your team on technical direction?** A: I create a structured architecture review where each side presents trade-offs with data — latency, cost, maintainability — rather than opinions. Usually the trade-off analysis itself resolves the disagreement; if not, I make the call and explain the reasoning.

**Q17. How do you prioritize competing roadmap demands from product and business leadership?** A: I map every ask against business impact and engineering cost, and make trade-offs visible — showing what gets delayed if we prioritize something else, so decisions are made with full information rather than by whoever asks loudest.

**Q18. How do you communicate technical concepts to non-technical stakeholders?** A: I lead with business outcomes first — "this reduces onboarding time by X" — then layer in technical detail only as needed. Diagrams and analogies help enormously, especially for something as abstract as agentic AI.

**Q19. Describe your leadership style.** A: Collaborative but decisive — I want input from the team before big decisions, but once we've discussed trade-offs, I make a clear call so we can move forward with speed.

**Q20. How do you manage global, distributed teams?** A: Clear async documentation of decisions, overlapping working hours for key syncs, and strong delegation to regional leads who can make day-to-day calls without waiting on me.

**Q21. How do you approach performance management for underperforming engineers?** A: Early, direct, private conversations focused on specific behaviors/outcomes, not character — followed by a concrete improvement plan with check-ins. I've found addressing issues early prevents them from becoming bigger problems.

**Q22. What's your approach to hiring?** A: I look for strong fundamentals in distributed systems thinking, curiosity about emerging tech like agentic AI, and how candidates reason through ambiguous problems — not just what they've memorized.

**Q23. How do you drive engineering execution across multiple teams/pods?** A: Regular architecture reviews, clear ownership per component (like our domain-agent model), and metrics that make progress and blockers visible early rather than at the end of a sprint.

**Q24. Tell me about a time you had to make an unpopular technical decision.** A: (Use a real example — e.g., enforcing stricter governance/guardrails on the agentic platform even though it added complexity, because of the regulated environment. Emphasize the reasoning and how you brought the team along.)

**Q25. How do you stay technically current while in a leadership role?** A: I stay hands-on with architecture reviews, actively follow developments in agentic AI and LLM platforms, and speak at conferences (PyTorch Conference Europe, MCP Dev Summit) — preparing talks forces me to go deep.

---

## Section 3: Agentic AI Platform — JPMorgan (Q26–40)



**Q26. Walk me through the Agentic AI platform you built at JPMorgan.** A: We converted marketing campaign onboarding — previously a manual, multi-system process — into a prompt-driven workflow. A business user describes what they need in natural language, and a chain-of-agents system (Intent Agent → Orchestrator → domain-specific agents) handles interpretation, planning, and execution across enterprise backend systems, all within governed guardrails.

**Q27. Why did you choose a multi-agent architecture over a single LLM call?** A: Separation of concerns. A single LLM handling everything is hard to audit, hard to scope permissions on, and hard to extend. Splitting into Intent, Orchestrator, and domain agents means each has a bounded responsibility, its own guardrails, and can be modified independently.

**Q28. What does the Intent Agent do specifically?** A: It parses the user's natural language request and converts it into a structured intent — identifying what the user wants to accomplish and extracting key parameters — before anything else in the pipeline runs.

**Q29. What does the Orchestrator Agent do?** A: It takes the structured intent and dynamically plans the workflow — deciding which domain agents need to be invoked, in what sequence, and how to handle dependencies between steps (e.g., campaign metadata before approval).

**Q30. How do domain-specific service agents work?** A: Each agent owns one bounded responsibility and integrates with a specific enterprise system — for example, one handles campaign metadata, another handles approvals/compliance checks, another integrates with marketing backend systems.

**Q31. How is this architecture "governed" — what does that mean in practice?** A: Every agent operates within explicitly scoped permissions, all actions are logged for audit, and the orchestrator enforces guardrails on what each domain agent can and cannot do. This is critical in banking, where every automated action needs to be traceable and compliant.

**Q32. How do you handle hallucination risk in this system?** A: By constraining what each agent can do — domain agents execute well-defined operations against real systems rather than freely generating actions, and the Intent Agent's output is structured/validated before the Orchestrator acts on it. (Add human-in-the-loop checkpoints if applicable to your real design.)

**Q33. How would you extend this platform to a new use case?** A: Because of the modular design, you'd add a new domain-specific agent for the new capability — the Intent and Orchestrator layers don't need to change, which is exactly the extensibility this architecture was designed for.

**Q34. What was the business impact of this platform?** A: Reduced manual effort in campaign onboarding and accelerated campaign launch timelines — turning a multi-system, multi-step manual process into a single conversational workflow.

**Q35. What were the biggest technical challenges in building this?** A: Balancing AI flexibility with enterprise governance — making the system flexible enough to handle varied

natural language requests while ensuring every action stayed within strict security and compliance boundaries required in banking.

**Q36. How did you ensure security in an AI-driven workflow handling enterprise data?**

A: Scoped access per agent, audit logging on every action, and architecture reviews with security/compliance stakeholders built into the design process from the start — not bolted on afterward.

**Q37. How do you measure success for an agentic AI system like this?** A: Reduction in manual effort/time-to-launch, adoption rate among business users, and system reliability metrics (successful completions, error rates, and audit compliance).

**Q38. What would you do differently if you rebuilt this platform today?** A: (Personalize — e.g., "I'd invest earlier in standardized observability across agents" or "I'd formalize the domain-agent interface contract sooner to speed up onboarding new agents.")

**Q39. How does this platform integrate with existing enterprise backend systems?** A: The domain agents act as an integration layer — each one is responsible for talking to a specific backend system (marketing platform, approval systems, data stores) using existing enterprise APIs, so the AI layer sits on top of, not instead of, existing infrastructure.

**Q40. What's the difference between "agentic AI" and traditional automation/RPA?** A: Traditional automation follows fixed, pre-scripted rules. Agentic AI dynamically interprets intent and plans variable workflows in response — the Orchestrator decides the steps at runtime based on context, rather than following a hardcoded sequence, making it far more adaptable to varied requests.

---

## Section 4: Real-Time Fraud Detection (Q41–50)

**Q41. Tell me about the fraud detection work you led.** A: I defined the architecture for real-time fraud detection services supporting card transactions, combining AI-based risk scoring with distributed systems design to evaluate risk within the transaction's critical path.

**Q42. Why is latency so critical in fraud detection?** A: Fraud decisions have to happen within the transaction flow itself — before it's approved or declined — so any added latency directly affects the customer experience and the bank's transaction throughput.

**Q43. How did you approach making inference fast enough for real-time transactions?**

A: Focusing on low-latency model serving, efficient data pipelines to deliver fresh signals to the model quickly, and horizontally scalable infrastructure to handle peak transaction volume without degrading response time.

**Q44. What's the trade-off between fraud detection accuracy and customer experience?**

A: Being overly aggressive blocks legitimate transactions (false positives), hurting customer trust; being too lenient lets fraud through. The architecture needs to support tunable thresholds so the business can calibrate that trade-off deliberately.

**Q45. How do you keep a fraud detection system resilient under high transaction volume?** A: Horizontally scalable, distributed service design, so throughput scales with load rather than becoming a bottleneck — this was central to the architecture I defined.

**Q46. What signals would you typically use in a fraud model?** A: (Keep general/conceptual given confidentiality) Transaction patterns, device and location context, and historical behavior signals — combined to produce a real-time risk score.

**Q47. How does this system integrate with existing risk/transaction infrastructure?** A: It sits within the transaction processing path, receiving transaction data, returning a risk decision within the required latency budget, and integrating with existing risk management and case-review systems.

**Q48. What would happen if the fraud detection service went down?** A: (Good answer if true) The architecture is designed with resilience/fallback in mind, since a full outage in the transaction path isn't acceptable — this is a key reason distributed systems principles were central to the design.

**Q49. How do you validate a fraud model is working well post-deployment?** A: Continuous monitoring of false positive/negative rates, feedback loops from confirmed fraud cases, and periodic model retraining/recalibration — treating this as a living system, not a one-time deployment.

**Q50. What's unique about doing fraud detection in a banking environment vs. e-commerce?** A: Regulatory scrutiny is much higher, transaction volumes and stakes are larger, and there's a much stronger requirement for explainability and auditability of every decision.

---

## Section 5: On-Device AI Inference — ExecuTorch (Q51–58)

**Q51. What is ExecuTorch, and why did you introduce it?** A: ExecuTorch is a framework for running AI models directly on-device (e.g., mobile) rather than server-side. I introduced it to improve latency, keep sensitive data on the device for privacy, and simplify regulatory compliance around data movement.

**Q52. What engineering challenges come with on-device inference vs. server-side?** A: You're constrained by the device's memory, compute, and battery — so models and inference pipelines need to be optimized/lightweight, unlike server-side where you have far more resources available.

**Q53. How did you handle model size/performance constraints on-device?** A: (Personalize with real approach — e.g., model optimization/quantization techniques, choosing lightweight architectures suited for mobile constraints.)

**Q54. What's the privacy benefit of on-device inference specifically?** A: Sensitive customer data never has to leave the device to get a prediction — it's processed locally, which reduces exposure and simplifies compliance with data residency and privacy regulations.

**Q55. How does on-device inference help with regulatory compliance?** A: Since data doesn't cross network/geographic boundaries for processing, it avoids much of the complexity around cross-border data transfer rules that server-side processing can trigger in banking.

**Q56. What trade-offs did you make between on-device and server-side inference?** A: On-device gains latency and privacy but trades off some model complexity/accuracy versus what you could run server-side with more compute — so it's best suited for use cases where speed and privacy outweigh needing the largest possible model.

**Q57. Which use cases at JPMorgan benefited most from on-device inference?** A: Areas needing immediate response and heightened privacy — this pattern is well suited to any latency- or privacy-sensitive use case, and is something we can extend to other parts of the platform going forward.

**Q58. How do you monitor/update models once they're deployed on-device?** A: (Personalize — e.g., versioned model deployment through app updates or over-the-air model updates, with monitoring on the server side for aggregate performance signals.)

---

## Section 6: IBM Watson Knowledge Catalog / Data Governance (Q59–75)

**Q59. Tell me about your work on IBM Watson Knowledge Catalog.** A: I led engineering and product teams building Watson Knowledge Catalog — an enterprise-scale data governance and catalog platform. I architected the data lineage and governance capabilities, improving data traceability by 70%, and delivered LLM-powered compliance and explainability features.

**Q60. What is data lineage, and why does it matter?** A: Data lineage tracks where data originates, how it moves and transforms across systems, and where it ends up being used. It matters because enterprises need to trace and justify how data was used — especially for regulatory compliance and debugging data quality issues.

**Q61. How did you improve data traceability by 70%?** A: (Personalize with specifics — e.g., architecting automated lineage capture across ingestion and transformation pipelines instead of relying on manual documentation, using metadata tagging integrated into the data pipeline itself.)

**Q62. What LLM-powered compliance/explainability capabilities did you build?** A: We used LLMs (including Mistral) to help automatically generate human-readable explanations

of data classifications and compliance decisions, making it easier for non-technical compliance teams to understand why data was tagged/governed a certain way.

**Q63. How did you improve customer satisfaction by 25%?** A: Through AI model monitoring and feedback systems — giving customers visibility into model behavior and creating a feedback loop so issues could be identified and addressed proactively rather than reactively.

**Q64. What tech stack did you use for this platform?** A: Java, Spring Boot, Kafka, GraphQL, Go, running on OpenShift, with DB2 as a data store, and Mistral LLM for AI-powered capabilities.

**Q65. Why did you choose Kafka for this platform?** A: For event-driven, large-scale enterprise data ingestion — Kafka gave us the throughput and decoupling needed to process data changes across many source systems reliably.

**Q66. Why GraphQL alongside REST?** A: GraphQL let API consumers request exactly the data they needed for complex, nested catalog/governance queries, reducing over-fetching, while REST remained useful for simpler, more standard integrations.

**Q67. How did you reduce infrastructure cost through data deduplication?** A: (Personalize — e.g., identifying duplicate data assets/metadata across the catalog and consolidating storage, reducing redundant processing and storage costs.)

**Q68. What does "data governance" mean in an enterprise AI context?** A: It's the set of processes and controls ensuring data is accurate, secure, compliant, and properly classified — which becomes even more critical when that data feeds AI models, since governance failures propagate directly into AI decision quality and compliance risk.

**Q69. How do you approach explainability in AI systems?** A: By designing for it from the start — capturing lineage and decision context as data flows through the system, so when an AI makes a decision, you can trace back exactly what data and logic contributed to it.

**Q70. What was your role — Product Owner and Architect — and how did you balance both?** A: As Product Owner, I owned prioritization and roadmap in partnership with business stakeholders; as Architect, I owned the technical design. Balancing both meant constantly translating business requirements into technical trade-offs and vice versa.

**Q71. How did you work with product and business stakeholders on this platform?** A: Regular roadmap reviews translating governance/compliance requirements into technical capabilities, and using customer feedback (via the monitoring systems we built) to prioritize what to build next.

**Q72. What's the biggest lesson you learned building a data governance platform?** A: Governance can't be an afterthought bolted onto data pipelines — it has to be architected in from the start (like lineage capture), or you end up trying to reconstruct history you never captured.

**Q73. How does this data governance experience apply to agentic AI governance?** A: Directly — the same principles (traceability, auditability, scoped access, explainability) that

governed data at IBM are exactly what I applied when designing the guardrails for the Agentic AI platform at JPMorgan.

**Q74. What was the scale of data/users this platform supported?** A: (Personalize with real scale figures if comfortable sharing — e.g., "supporting large enterprise customers with data volumes in the [X] range.")

**Q75. How did you handle performance at scale with DB2 and Kafka together?** A: (Personalize — e.g., using Kafka for asynchronous, high-throughput ingestion while DB2 handled durable, queryable storage of catalog metadata, keeping the write path decoupled from the query path.)

---

## Section 7: OpenText — AI SaaS Platform (Q76–85)

**Q76. Tell me about your role at OpenText.** A: I led delivery of a cloud-native AI SaaS platform, managing a team of eight engineers, defining the product roadmap and architecture with Product and Design, and building GenAI-powered retrieval services and developer productivity tooling.

**Q77. What does "GenAI-powered retrieval services" mean in this context?** A: Building retrieval-augmented capabilities — using GenAI to help users find and synthesize relevant information from large data sets more effectively than traditional search.

**Q78. What tech stack did you use at OpenText?** A: Java 17, Spring Boot, AWS, Camunda for workflow orchestration, and PostgreSQL.

**Q79. Why Camunda for workflow orchestration?** A: Camunda gave us a robust, visual way to model and manage business process workflows, which was useful for orchestrating multi-step SaaS platform processes reliably.

**Q80. What did "modernizing engineering delivery" involve at OpenText?** A: Introducing stronger AWS-based CI/CD pipelines, observability tooling, and DevOps best practices — moving the team toward faster, safer, more visible releases.

**Q81. How did you manage a team of eight while also owning architecture?** A: I delegated day-to-day execution to sub-leads within the team while retaining ownership of key architecture decisions and roadmap alignment with Product and Design.

**Q82. What AI developer productivity capabilities did you build?** A: (Personalize — e.g., tools that used GenAI to help developers generate code, documentation, or tests faster within the SaaS platform's development workflow.)

**Q83. Why did you leave OpenText for JPMorgan?** A: (Personalize — e.g., "The opportunity to lead AI platform architecture at much larger enterprise scale, in a regulated environment where governance and AI intersect even more deeply.")

**Q84. What was most challenging about building a cloud-native AI SaaS platform?** A: Balancing GenAI's inherent unpredictability with the reliability and consistency SaaS customers expect — required careful architecture around retries, fallbacks, and observability specific to AI-driven components.

**Q85. How is this experience relevant to future AI platform roles?** A: It gave me hands-on experience owning both product roadmap and technical architecture for a GenAI product end-to-end — a foundation I built directly on at JPMorgan with the Agentic AI platform.

---

## **Section 8: Earlier Career (TCS, eGain, Pramati, ADP) (Q86–92)**

**Q86. Tell me about your time at Tata Consultancy Services.** A: I started my career there as a Systems Engineer, developing enterprise web applications using Java and Spring — it gave me a strong foundation in enterprise software fundamentals.

**Q87. What did you work on at eGain Communications?** A: I developed enterprise knowledge management applications, which — looking back — connects directly to the data/knowledge platform work I later did at IBM.

**Q88. Tell me about your role at Pramati Technologies.** A: I developed microservices and resiliency frameworks for analytics platforms — early exposure to distributed systems design principles I've used throughout my career since.

**Q89. What did you do as a Consultant at ADP?** A: I designed extensible Java platform architecture for next-generation HR solutions — my first real ownership of architectural decisions at scale.

**Q90. How did these early roles shape your approach to architecture today?** A: They gave me ground-level experience with the failure modes of poorly designed systems — which is why I emphasize modularity, resiliency, and extensibility so strongly in my architecture decisions today.

**Q91. What's the biggest technical skill gap you've closed since your early career?** A: (Personalize — e.g., "Moving from traditional enterprise Java architecture into deep AI/ML systems design — I invested heavily in understanding GenAI and agentic architectures over the last few years.")

**Q92. How has enterprise software architecture changed since you started your career?** A: The shift from monolithic to microservices/event-driven architecture, and more recently, the addition of AI/agentic components as first-class architectural elements — not just bolted-on features.

---

## Section 9: General Technical & Architecture Questions (Q93–97)

**Q93. How do you approach designing a distributed system from scratch?** A: Start with the failure modes — what happens when a component is slow or down — then design for resiliency, scalability, and observability from day one, not as an afterthought. I also prioritize clear service boundaries, much like the domain-agent separation in the Agentic AI platform.

**Q94. How do you decide between REST, GraphQL, and event-driven (Kafka) integration patterns?** A: REST for simple, standard request/response integrations; GraphQL when consumers need flexible, nested queries without over-fetching; Kafka/event-driven when you need decoupled, asynchronous, high-throughput processing across many systems.

**Q95. What's your approach to system observability?** A: Instrumenting for logs, metrics, and traces from the start, with clear SLAs/alerting tied to business-critical paths — especially important in AI systems where you also need to observe model behavior, not just infrastructure health.

**Q96. How do you approach security architecture for AI systems specifically?** A: Treating each AI component (agent, model) as a scoped, permissioned entity rather than a trusted black box — enforcing least-privilege access and audit logging, the same principles I applied in the Agentic AI platform's governance layer.

**Q97. What excites you most about the future of agentic AI in enterprise settings?** A: The ability to safely automate complex, judgment-involving workflows — not just repetitive tasks — while keeping humans and governance in the loop. That combination of capability and control is what makes it enterprise-ready.

---

## Section 10: Behavioral & Closing Questions (Q98–100)

**Q98. Tell me about a time you failed and what you learned.** A: (Use a real, specific example — ideally technical or leadership related. Structure: situation, what went wrong, what you learned, how you've applied it since. Avoid generic/rehearsed-sounding answers.)

**Q99. Why should we hire you?** A: I bring a rare combination — deep architecture experience with distributed and AI systems, proven delivery of agentic AI at enterprise scale in a highly regulated environment, and 9+ years of engineering leadership managing global teams. I don't just design AI systems conceptually — I've shipped them in production with real governance and compliance constraints.

**Q100. Do you have any questions for us?** A: (Always have 2–3 ready — examples: "What does success look like in this role in the first 6 months?" / "How is your team currently thinking about governance for AI-driven workflows?" / "What's the biggest technical challenge the team is facing right now?")



---

## Prep tips

- Personalize every "(Personalize...)" placeholder with your real specifics before the interview — panels probe for authenticity.
- For confidential/proprietary details (exact latency numbers, model architectures, fraud signals), it's fine to speak at a conceptual level — panels understand banking constraints.
- Practice the architecture questions (Q26–40) out loud with the diagram in mind — being able to sketch or verbally walk through the chain-of-agents flow smoothly will stand out.